

Project Sprint #1

The Solitaire game is described in CS449HomeworkOverview. You should read the description very carefully.

In this assignment, you aim to specify the requirements (i.e., user stories and acceptance criteria) of the target software that allows a human player to play a Solitaire game. These requirements will be fully implemented by the end of sprint 3. The minimum features include **choosing the board size and type, starting a new game, making a move, determining if a game is over**. The following is a sample GUI layout.

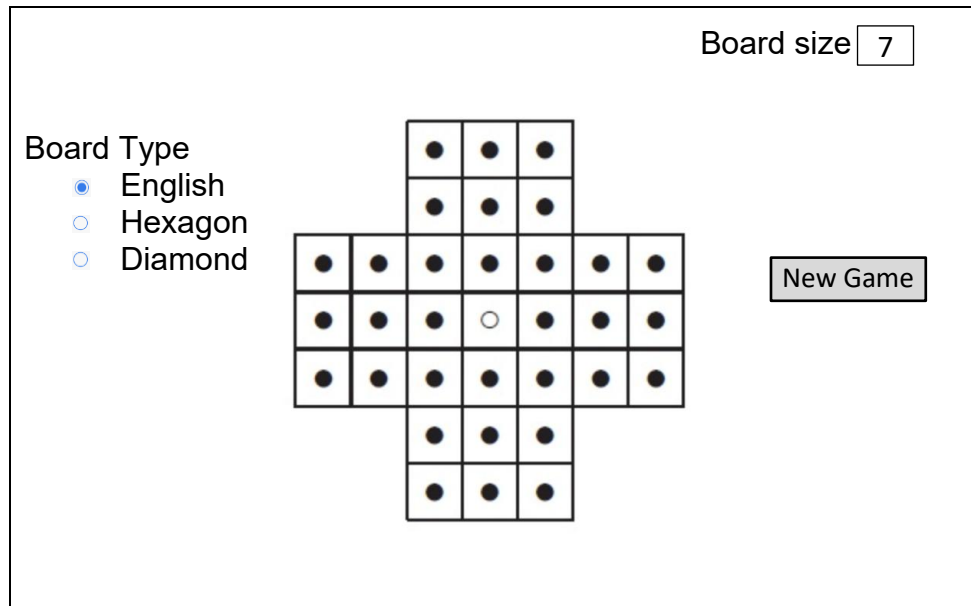


Figure 1. Sample GUI layout of the first working program by the end of Sprint 3

Use the following tables to document your user stories and acceptance criteria.

You are required to use the free ChatGPT or another LLM to complete two user stories and two of their respective acceptance criteria. You must ensure that the generated user stories are correct and fix them if not. At the end of the submission, provide screenshots of your LLM prompts and answers, along with the description of errors the LLM made that you had to correct. Points will be deducted if no screenshots/description are provided.

For the other user stories and acceptance criteria, you are not required to use LLM.

I. User Stories (3 points)

- **User Story Template:** As a <role>, I want <goal> [so that <benefit>]

ID	User Story Name	User Story Description	Priority	Estimated effort (hours)
1	Choose a board size	As a player, I want to select a board size so that I can customize my game experience through board size.	A	2
2	Choose the board type	As a player, I want to select a board type so that I can customize my game experience through board type.	A	2
3	Start a new game of the chosen board size and type	As a player, I want to start a new game with the currently selected board size and type so that I can play the game as I intended.	B	1

ID	User Story Name	User Story Description	Priority	Estimated effort (hours)
4	Make a move in a game	As a player, I want to move a peg by jumping over another peg into an empty space so that I can remove pegs and progress through the game.	C	4
5	A game is over	As a player, I want the game to determine when the game is over so that I know whether I have won or lost.	C	3

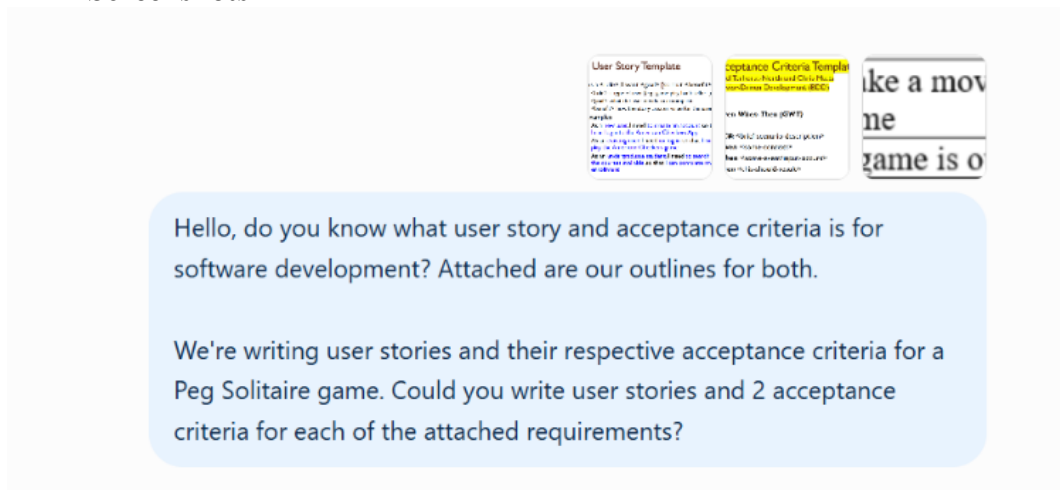
II. Acceptance Criteria (AC) (12 points): Add/delete rows as needed.

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inProgress)
1. Choose a board size	1.1	<i>AC 1.1 Customizing a board size</i> <u>Given</u> that the program is open <u>When</u> a player selects the Board Size drop down menu and chooses a board size <u>Then</u> that board size will be configured on the screen <u>And</u> all valid board positions will contain pegs except the center position, which should be empty	toDo
	1.2	<i>AC 1.2 Default board size</i> <u>Given</u> that the program is not yet open <u>When</u> a player first opens the program <u>Then</u> the board size defaults to 7 <u>And</u> all valid board positions will contain pegs except the center position, which should be empty	toDo
2. Choose the board type	2.1	<i>AC 2.1 Customizing the board type</i> <u>Given</u> that the program is open <u>When</u> a player selects the Board Type drop down menu and chooses a board type <u>Then</u> that board type will be configured on the screen <u>And</u> all valid board positions will contain pegs except the center position, which should be empty	toDo
	2.2	<i>AC 2.2 Default board type</i> <u>Given</u> that the program is not yet open <u>When</u> a player first opens the program <u>Then</u> the board type defaults to English <u>And</u> all valid board positions will contain pegs except the center position, which should be empty	toDo
3. Start a new game of the chosen board size and type	3.1	<i>AC 3.1 Starting the game</i> <u>Given</u> that the program is open <u>When</u> a player selects "Start Game" <u>Then</u> the game begins with the currently configured board size, type and pegs	toDo
4. Make a move in a game	4.1	<i>AC 4.1 Player makes a valid move</i> <u>Given</u> the player is playing the game <u>And</u> the player has selected a peg that can jump orthogonally or diagonally over an adjacent peg into an empty space two positions away <u>When</u> the player clicks on the empty space <u>Then</u> the selected peg should move to the empty space and the jumped peg should be removed	toDo
	4.2	<i>AC 4.2 Player attempts an invalid move</i> <u>Given</u> the player is playing the game <u>And</u> the player selects a move that does not follow the rules of Peg Solitaire <u>When</u> the player attempts to make the move <u>Then</u> the move should not be performed and the board should remain unchanged	toDo

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inProgress)
	4.3	<i>AC 4.3 Undo a move</i> <u>Given</u> the player is playing the game <u>And</u> the player has already made one or more valid moves <u>When</u> the player clicks the undo button <u>Then</u> the most recent move will be undone and the board will return to the state it was in before that move	toDo
5. A game is over	5.1	<i>AC 5.1 Player wins the game</i> <u>Given</u> the player is playing a game <u>When</u> there is only one peg remaining on the board <u>Then</u> the system should end the game <u>And</u> display the player's rating as Outstanding	toDo
	5.2	<i>AC 5.2 Player has no valid moves remaining</i> <u>Given</u> the player is playing a game <u>When</u> there are no more valid moves left <u>Then</u> the system should end the game <u>And</u> display the player's rating based on the number of pegs remaining: Very Good for 2, Good for 3, and Average for 4 or more	toDo

*LLM used for User Stories 4 and 5, and Acceptance Criterion 4.1, 4.2, 5.1. 5.2.

LLM Screenshots:



Requirement 4: Make a move in a game

User Story:

As a player, I want to move a peg by jumping over another peg into an empty space so that I can remove pegs and progress through the game.

AC1: Make a valid move

Given the player has selected a peg with an adjacent peg and an empty space directly beyond it,
When the player moves the selected peg into the empty space,
Then the selected peg should move to the empty space and the jumped peg should be removed.

AC2: Attempt an invalid move

Given the player selects a move that does not follow the rules of Peg Solitaire,

When the player attempts to make the move,

Then the move should not be performed and the board should remain unchanged.

Requirement 5: A game is over

User Story:

As a player, I want the game to determine when the game is over so that I know whether I have won or lost.

AC1: Player wins the game

Given there is only one peg remaining on the board,

When the game checks whether the game is over,

Then the game should end and inform the player that they have won.

AC2: Player has no valid moves remaining

Given more than one peg remains on the board and there are no valid moves available,

When the game checks whether the game is over,

Then the game should end and inform the player that they have lost.

These are especially nice acceptance criteria because they're **testable**: you can actually set up a board in each condition and verify whether the program behaves as specified.

The LLM didn't make any "errors" per say; there were just some parts that needed tweaking (and me and the LLM worked together to tweak it). For example, we added more detail to AC 4.1: Player makes a valid move. Specifically, what criteria must be met for a move to be considered valid. I also took off the "When the game checks whether the game is over" criteria on AC 5.1 and 5.2; that wording seemed odd. I just moved the "Given" conditions to the "When" portion and made the Given condition as "the player is playing the game". I added that condition to many other acceptance criteria as well, since playing the game / having started the game is a necessary pre-condition for many of them. I also changed AC 5.1 and 5.2 to display the player's *rating* after the game has ended, not just whether the player has "won" or "lost", as per the instructions.